

## How Docker Swarm evolved

In early 2014, Docker developed a cluster management system with a communication protocol known as Beam. Later, with the Docker API, a daemon was introduced to communicate with a distributed system. This was named Libswarm. The daemon is called Swarmd.

In November 2014, the Docker team retained the concept of cluster communication with additional remote APIs and named this Swarm. This first generation was called Swarm v1.

In February 2016, Swarm v1 was completely redesigned to overcome certain limitations and was named Swarm v2. In June 2016, SwarmKit, which is an orchestration toolkit within the Docker engine for distributed service, came into being. This version is also known as 'Swarm mode'.

## Features of Docker Swarm

The features of Docker Swarm are:

- **Decentralised design:** You can manage the nodes in the Swarm cluster through Swarm commands. This gives a single point of access to build the entire Swarm.
  - **Scaling:** You can specify the number of tasks for every service. This is automatically done through commands to scale the service.
  - **Desired state reconciliation:** The Swarm manager keeps track of the cluster state so that the actual and the desired state is always the same.
  - **Multi-host networking:** When you specify an overlay network to connect your services, the Swarm manager assigns addresses to the containers on the overlay network once you create/update the containers.
  - **Service discovery:** Swarm manager nodes allocate a unique DNS name for the services. You will be able to find containers in the swarm through the DNS server embedded in the swarm.
  - **Load balancing:** SwarmKit has an internal load balancer which distributes the service containers within nodes. You can include an external load balancer as well.
- Before we dive into the architecture of Swarm, let's look

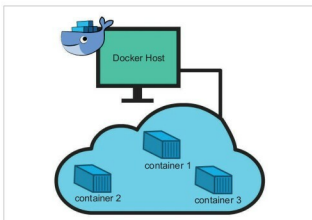


Figure 1: The Docker network

at some of the key concepts in Docker Swarm.

- Docker node
- Docker service
- Docker tasks

## Docker Node

Docker Node is a Docker engine instance that is included in Docker Swarm. In real-time, these nodes are distributed across multiple clouds as well as physical machines.

There are two kinds of Docker nodes:

- Manager node
- Worker node

**Manager node:** The manager node is responsible for all the orchestration and container management activities required to keep up the desired system state.

**Worker node:** The worker node executes the tasks assigned by the manager node.

## Docker service

Service is nothing but a task definition that has to be executed. You will have to create a service specifying the image name and other additional parameters. In most cases, service is an image for a microservice of some large application. An example would be an HTTP server or a database, any kind of executable program that you would run in a distributed system, etc.

## Architecture

The node is the key member of a Docker Swarm. A Swarm can have more than one manager node. In this case, the nodes elect their leader using the Raft algorithm to conduct the orchestration tasks.

Manager nodes are also worker nodes. But you can configure them as manager-only nodes, thereby restricting them from working on any other tasks.

The Docker Swarm API is another key component of the Docker Swarm architecture.

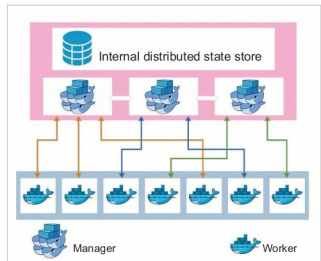


Figure 2: Docker Swarm architecture